

07 | 类型与类型参数：给Rust小助手提供更多信息

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们一起学习 Rust 中类型相关的知识。

这块儿知识在其他大部分语言入门材料中讲得不多，但是对于 Rust 而言，却是非常重要而有趣的。我们都知道，计算机硬件执行的代码其实是二进制序列。而**对一个二进制值来说，正是类型赋予了它意义。**

比如 01100001 这个二进制数字，同样的内存表示，如果是整数，就表示 97。如果是字符，就表示 'a' 这个 char。如果没有类型去赋予它额外的信息，当你看到这串二进制编码时，是不知道它代表什么的。

类型

《Programming.with.Types》2019 这本书里对类型做了一个定义，翻译出来是这样的：类型是对数据的分类，这个分类定义了这些数据的意义、被允许的值的集合，还有能在这些数据

上执行哪些操作。编译器或运行时检查类型化过程，以确保数据的完整性，对数据施加访问限制，以及把数据按程序员的意图进行解释。

有些情况下，我们会简化讨论，把操作部分忽略掉，所以我们可以简单地把类型看作集合，这个集合表达了这个类型的实例能取到的所有可能的值。

类型系统

这本书里还定义了类型系统的概念。

书里是这样说的：类型系统是一套规则集——把类型赋予和施加到编程语言的元素上。这些元素可以是变量、函数和其他高阶结构。类型系统通过你在代码中提供的标注来给元素赋予类型，或者根据它的上下文隐式地推导某个元素的类型。类型系统允许在类型间做各种转换，同时禁止其他的一些转换。

举例来说，刚刚我们提到的类型的标注就像这种 `let a: u32 = 10;`。我们用 `: u32` 这种语法对变量 `a` 进行了标注，表明变量 `a` 的类型是 `u32` 类型。`u32` 可以转换成 `u64`。

```
1 let b = a as u64;
```

 复制代码

但是 `u32` 不能直接转换到 `String` 上去。

```
1 fn main() {  
2     let a: u32 = 10;  
3     let b = a as String;    // 错误的  
4  
5     println!("{b}");  
6 }
```

 复制代码

类型化的好处

类型化有 5 大好处：正确性、不可变性、封装性、组合性、可读性。这 5 大好处中的每一个都是软件工程理论推崇的。

Rust 语言非常强调**类型化**，它的类型系统非常严格，隐式转换非常少，在某些简单场景下甚至会引起初学者的不适。

比如下面这个代码：

 复制代码

```
1 fn main() {  
2     let a = 1.0f32;  
3     let b = 10;  
4  
5     let c = a * b;  
6 }
```

编译错误，提示你不能将一个浮点数和一个整数相乘。

 复制代码

```
1 error[E0277]: cannot multiply `f32` by `{integer}`  
2 --> src/main.rs:5:15  
3 |  
4 5 |     let c = a * b;  
5 |               ^ no implementation for `f32 * {integer}`  
6 |  
7 = help: the trait `Mul<{integer}>` is not implemented for `f32`  
8 = help: the following other types implement trait `Mul<Rhs>`:  
9     <&'a f32 as Mul<f32>>  
10    <&f32 as Mul<&f32>>  
11    <f32 as Mul<&f32>>  
12    <f32 as Mul>
```

初学者遇到这种情况，往往会觉得 Rust 过于严苛，一个很自然的操作都不让我通过，烦死了，马上就想放弃了。

当遇到这种基础类型转换错误时，可以尝试使用 **as 操作符** 显式地将类型转成一致。

修改上述代码如下：

[复制代码](#)

```
1 fn main() {  
2     let a = 1.0f32;  
3     let b = 10 as f32;    // 添加了 as f32  
4  
5     let c = a * b;  
6 }
```

这段代码就可以编译通过了。

这里其实展示出 Rust 的一个非常明显的特点：**尽可能地显式化**。显式化包含两层意思。

1. 不做自动隐式转换。
2. 没有内置转换策略。

不做自动隐式转换，可以这样来理解，比如前面的示例，当别人来看你的代码的时候，多了 `as f32` 这几个字符，他就明白你是在做类型转换，就会自然地警觉起来，分析上下文，估计出下面算出的结果是什么类型。这相当于由程序员为编译器明确地提供了一些额外的信息。

没有内置转换策略这一点，我们可以拿 JavaScript 社区中流传的一张梗图来对比说明。


```

> typeof NaN           > true==1
< "number"            < true

> 9999999999999999    > true===1
< 100000000000000000 < false

> 0.5+0.1==0.6        > (!+[[]+[]+![]).length
< true                < 9

> 0.1+0.2==0.3        > 9+"1"
< false              < "91"

> Math.max()          > 91-"1"
< -Infinity          < 90

> Math.min()          > []==0
< Infinity           < true

> []+[]
< ""

> []+{}
< "[object Object]"

> {}+[]
< 0

> true+true+true===3
< true

> true-true
< 0

```



图片来源：<https://www.codenong.com/cs107029540/>

取图里的一个示例，在 JavaScript 里，`9 + "1"` 计算出来的结果是 `"91"`。这其实就涉及两个不同类型之间相加如何处理的问题。在这个问题上，JavaScript 自己内置了策略，把两个不同类型的值加起来，硬生生算出了一个结果。而当遇到 `91 - "1"` 时，策略又变了，算出了数字 90。这些就是内置类型转换策略的体现。

而在 Rust 中，`9+"1"` 是不可能通过编译的，更不要说计算出一个神奇的结果了。如果要写出类似的代码，在 Rust 中可以这样做。

```
1 fn main() {  
2     let a = 9 + '1' as u8;  
3     let b = 9.to_string() + "1";  
4 }
```

[复制代码](#)

有没有觉得特别清晰！我们一眼就能推断出 a 和 b 的类型，a 为 u8 类型，b 为 String 类型。

这就是 Rust 的严谨，它有着严密的类型体系，在类型化上绝不含糊。它从底层到上层构建了一套完整严密的类型大厦。你的项目越大，使用 Rust 感觉也就越舒服，原因之一就是严谨的类型系统在为你保驾护航。

类型作为一种约束

前面提到，类型是变量所有可能取得的值的集合。换句话说，类型实际上限制或定义了变量的取值空间。因此，**类型对于变量来说，也是一种约束。**

实际上，Rust 中的：（冒号）在语法层面上就是约束。

示例：

```
1 let a: u8 = 10;  
2 let b: String = "123".to_string();
```

[复制代码](#)

上述示例中，变量 a 被限制为只能在 u8 这个类型的值空间取值，也就是 0 到 255 这 256 个整数里的一个，而 10 属于这个空间。变量 b 被限制为只能在字符串值空间内取值。不管字符串的值空间多大（其实是无限），这些值与 u8、u32 这些类型的值也是不同的。

关于：作为约束的体现，我们会在后面的课程中不断看到。

多种类型如何表示？

前面我们讲到了，用一种类型来对变量的取值空间进行约束。这确实非常好，有利于程序的健壮性。但有时也会遇到这种方式不够用的场景。比如在 Rust 中，我们把整数分成 u8、u16、u32、u64。现在我想写一个函数，它的参数支持整数，也就是说要同时能接受 u8、u16、u32、u64 这几种类型的值，应该怎么办？如果只是采用前面的理论，这个需求是没法做到的。

再看另外一个实际的例子，我有一个日志函数，可以给这个函数传入数字、字符串等类型。这种需求很常见。如何让这一个日志函数同时支持数字和字符串作为参数呢？这就很头痛了。

这里实际提出了这样一个问题：**在 Rust 语言中，有没有办法用某种方式来表示多种类型？**答案是有的。

类型参数

在 Rust 语言中定义类型的时候，可以使用**类型参数**。比如标准库里常见的 `Vec<T>`，就带一个类型参数 T，它可以支持不同类型的列表，如 `Vec<u8>`、`Vec<u32>`、`Vec<String>` 等。这里这个 T 就表示一个类型参数，在定义时还不知道它具体是什么类型。只有在使用的時候，才会对这个 T 赋予一个具体的类型。

这里这个 `Vec<T>`，是一个类型整体，单独拆开来讲 Vec 类型是没有意义的。T 是 `Vec<T>` 中的类型参数，它其实也是信息，提供给 Rust 编译器使用。而带类型参数的类型整体（比如 `Vec<T>`）就叫做**泛型**（generic type）。

结构体中的类型参数

我们来看一个例子，自定义一个结构体 `Point<T>`。

```
1 struct Point<T> {  
2     x: T,  
3     y: T,  
4 }
```

 复制代码

这是一个二维平面上的点，由 x、y 两个坐标轴来定义。因为 x、y 的取值有可能是整数，也有可能是浮点数，甚至有可能是其他值，比如无穷精度的类型。所以我们定义一个类型参数 T，定义的时候需要把 T 放在结构体类型名字后面，用 `<>` 括起来，也就是 `struct Point<T>`。这里的 `Point<T>` 整体就成为了泛型。然后，标注 x 和 y 分量的类型都是 T。可以看到，T 占据了冒号后面定义类型的位置。所以说它是**占位类型**也没有问题。

对这个结构体的例子来说，其实还隐藏了一个很重要的细节：x 和 y 字段的类型都是 T，意味着 x 和 y 两个分量的类型是一样的。

我们来看这个 Point 结构体类型如何实例化。

[复制代码](#)

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 fn main() {
7     let integer = Point { x: 5, y: 10 };    // 一个整数point
8     let float = Point { x: 1.0, y: 4.0 };  // 一个浮点数point
9 }
```

符合我们预期，正常编译通过。那如果实例化的时候，给 x 和 y 赋予不同的类型值会怎样呢？我们来试试。

[复制代码](#)

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 fn main() {
7     let wont_work = Point { x: 5, y: 4.0 };
8 }
9
10 $ cargo run
11     Compiling chapter10 v0.1.0 (file:///projects/chapter10)
12 error[E0308]: mismatched types
13 --> src/main.rs:7:38
```

```

14 |
15 7 |     let wont_work = Point { x: 5, y: 4.0 };
16 |                                     ^^^ expected integer, found floating-poi

```

编译器正确地指出了问题，说期望整数却收到了浮点数，所以不通过。

这里有个细节，那就是编译器对 `Point<T>` 中的 `T` 参数进行了推导，因为它首先遇到的是 `x` 的值 `5`，是一个整数类型，因此编译器就把 `T` 具体化成了整数类型（具体哪一种还没说，不过在这里不重要），当再收到 `y` 分量的值的时候，发现是浮点数类型，和刚才的整数类型不一致了。而 `Point` 中定义的时候，又要求 `x` 和 `y` 的类型是相同的，这个时候就产生了冲突，于是报错。

那么，如何解决这个问题呢？Rust 并没有限制我们只能定义一个类型参数呀！定义多个就好了，把 `x` 分量和 `y` 分量定义成不同的参数化类型。

[复制代码](#)

```

1 struct Point<T, U> {
2     x: T,
3     y: U,
4 }
5
6 fn main() {
7     let both_integer = Point { x: 5, y: 10 };
8     let both_float = Point { x: 1.0, y: 4.0 };
9     let integer_and_float = Point { x: 5, y: 4.0 };
10 }

```

像这样，代码就可以顺利通过编译了。

再次强调，前面两个示例里的 `Point<T>` 和 `Point<T, U>` 都是一个类型整体。这个时候，你把 `Point` 这个符号本身单独拿出来是没有意义的。

在使用的时候，可以用 turbofish 语法 `::<>` 明确地给泛型，或者说是给 Rust 编译器提供类型参数信息，我们修改一下上面两个示例。

 复制代码

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 fn main() {
7     let integer = Point::<u32> { x: 5, y: 10 };
8     let float = Point::<f32> { x: 1.0, y: 4.0 };
9 }
```

 复制代码

```
1 struct Point<T, U> {
2     x: T,
3     y: U,
4 }
5
6 fn main() {
7     let both_integer = Point::<u32, u32> { x: 5, y: 10 };
8     let both_float = Point::<f32, f32> { x: 1.0, y: 4.0 };
9     let integer_and_float = Point::<u32, f32> { x: 5, y: 4.0 };
10 }
```

注意，使用时提供类型参数信息用的是`::<>`，而定义类型参数的时候只用到`<>`，注意它们的区别。Rust 把定义和使用两个地方通过语法明确地区分开了，而有的语言并没有区分这两个地方。

到这里，我们会体会到，类型参数存在两个过程，**一个是定义时，一个是使用时**。这两个过程的区分很重要。这里所谓的“使用时”，仍然是在编译期进行分析的，也就是分析你在代码的某个地方用到了这个带类型参数的类型，然后把这个参数具体化，从而形成一个最终的类型版本。

比如 `Point<T>` 类型的具化类型可能是 `Point<u32>`、`Point<f32>` 等等；`Point<T, U>` 类型的具化类型可能是 `Point<u32, u32>`、`Point<u32, f32>`、`Point<f32, u32>`、`Point<f32, f32>` 等等。到底有多少种具化版本，是看你在后面代码使用时，会用到多少种不同的具体类型。这个数目是由编译器自动帮我们计算展开的。

这种在编译期间完成的类型展开成具体版本的过程，被叫做**编译期单态化**。单态化的意思就是把处于混沌未知的状态具体化到一个单一的状态。

在泛型上做 impl

当类型是一个泛型时，要对其进行 impl 的话，需要处理类型参数，形式相比于我们在 [第 5 讲讲到的 impl 结构体](#)，有一点变化。这里我们还是用 `Point<T>` 举例。

[复制代码](#)

```
1 struct Point<T> {  
2     x: T,  
3     y: T,  
4 }  
5  
6 impl<T> Point<T> {           // 注意这一行  
7     fn play(n: T) {}         // 注意这一行  
8 }
```

在对 `Point<T>` 做 impl 的时候，需要在 impl 后面加一个 `<T>`，表示在 impl 的 block 中定义类型参数 `T`，供 impl block 中的元素使用，这些元素包括：`impl<T> Point<T>` 里 `Point<T>` 中的 `T` 和整个 impl 的花括号 body 中的代码，如 `play()` 函数的参数就用到了这个 `T`。

有一个细节需要注意：`struct Point<T>` 里 `Point<T>` 中的 `T` 是定义类型参数 `T`，`impl<T> Point<T>` 中的 `Point<T>` 中的 `T` 是使用类型参数 `T`，这个 `T` 是在 impl 后面那个尖括号中定义的。

在对泛型做了 impl 后，对其某一个具化类型继续做 impl 也是可以的，比如：

[复制代码](#)

```
1 struct Point<T> {  
2     x: T,  
3     y: T,  
4 }  
5  
6 impl<T> Point<T> {
```

```
7     fn play(n: T) {}  
8 }  
9  
10 impl Point<u32> {           // 这里，对具化类型 Point<u32> 继续做 impl  
11     fn doit() {}  
12 }
```

下面我们来看具体场景中的类型参数的使用。

枚举中的类型参数

前面讲过，枚举的变体可以挂载任意其他类型作为负载。因此每个负载的位置，都可以出现类型参数。比如最常见的两个枚举，`Option<T>` 与 `Result<T, E>`，就是泛型。

`Option<T>` 用来表示有或无。

[复制代码](#)

```
1 enum Option<T> {  
2     Some(T),  
3     None,  
4 }
```

`Result<T, E>` 用来表示结果，正确或错误。Ok 变体带类型参数 T，Err 变体带类型参数 E。

[复制代码](#)

```
1 enum Result<T, E> {  
2     Ok(T),  
3     Err(E),  
4 }
```

我们后面会详细阐述 `Option<T>` 和 `Result<T, E>` 的用法。

现在再看一个更复杂的枚举中带类型参数的例子。

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 enum Aaa<T, U> {
7     V1(Point<T>),
8     V2(Vec<U>),
9 }
```

我们来解释一下，枚举 `Aaa<T, U>` 的变体 `V1` 带了一个 `Point<T>` 的负载，变体 `V2` 带了一个 `Vec<U>` 的负载。由于出现了两个类型参数 `T` 和 `U`，所以需要在 `Aaa` 后面的尖括号里定义这两个类型参数。

实际上，类型参数也是一种复用代码的方式，可以让写出的代码更紧凑。下面我们来看具体的应用场景。

函数中的类型参数

需求是这样的：很多不同的类型，其实它们实现某个逻辑时，逻辑是一模一样的。因此如果没有类型参数，就得对每个具体的类型重新实现一次同样的逻辑，这样就显得代码很臃肿。重复的代码也不好维护，容易出错。

示例：

```
1 struct PointU32 {
2     x: u32,
3     y: u32,
4 }
5
6 struct PointF32 {
7     x: f32,
8     y: f32,
9 }
10
11 fn print_u32(p: PointU32) {
12     println!("Point {}, {}", p.x, p.y);
13 }
```

```
13 }
14
15 fn print_f32(p: PointF32) {
16     println!("Point {}, {}", p.x, p.y);
17 }
18
19 fn main() {
20     let p = PointU32 {x: 10, y: 20};
21     print_u32(p);
22
23     let p = PointF32 {x: 10.2, y: 20.4};
24     print_f32(p);
25 }
```

上面示例中，因为我们没有使用类型参数，那就得针对不同的字段类型（u32，f32）分别定义结构体（PointU32，PointF32）和对应的打印函数（print_u32，print_f32），并分别调用。

而有了类型参数的话，这样的需求代码只需要写一份，让编译器来帮我们分析到时候要应用到多少种不同的类型上。

上面的代码可以优化成这样：

[📄 复制代码](#)

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 fn print<T: std::fmt::Display>(p: Point<T>) {
7     println!("Point {}, {}", p.x, p.y);
8 }
9
10 fn main() {
11     let p = Point {x: 10, y: 20};
12     print(p);
13
14     let p = Point {x: 10.2, y: 20.4};
15     print(p);
16 }
```

是不是清爽多了！

实际上，清爽只是我们看到的样子。在编译的时候，Rust 编译器会帮助我们在这种泛型代码展开成前面那个示例的样子。这种脏活累活，Rust 编译器帮我们完成了。

细心的你可能发现了，`print` 函数的类型参数在定义的时候，多了一个东西。

[📄 复制代码](#)

```
1 fn print<T: std::fmt::Display>(p: Point<T>) {
```

这里 `T: std::fmt::Display` 的意思是要求 `T` 满足某些条件 / 约束。这里具体来说就是 `T` 要满足可以被打印的条件。因为我们这个函数的目的是把 `x` 和 `y` 分量打印出来，那么它确实要能被打印才行，比如得能转换成人类可见的某种格式。

关于这种约束，我们后面会详细讲述。

方法中的类型参数

结构体中可以有类型参数，函数中也可以有类型参数，它们组合起来，方法上当然也可以有类型参数。

示例：

[📄 复制代码](#)

```
1 struct Point<T> {
2     x: T,
3     y: T,
4 }
5
6 impl<T> Point<T> {           // 在impl后定义impl block中要用到的类型参数
7     fn x(&self) -> &T {     // 这里，在方法的返回值上使用了这个类型参数
8         &self.x
9     }
10 }
11
12 fn main() {
```

```

13     let p = Point { x: 5, y: 10 };
14     println!("p.x = {}", p.x());
15 }
16 // 输出
17 p.x = 5

```

上面的示例中，`Point<T>` 的方法 `x()` 的返回值类型就是 `&T`，使用到了 `impl<T>` 这里定义的类型参数 `T`。

下面我们继续看更复杂的内容，方法中的类型参数和结构体中的类型参数可以不同。

[复制代码](#)

```

1  struct Point<X1, Y1> {
2      x: X1,
3      y: Y1,
4  }
5
6  // 这里定义了impl block中可以使用的类型参数X3, Y3,
7  impl<X3, Y3> Point<X3, Y3> {
8      // 这里单独为mixup方法定义了两个新的类型参数 X2, Y2
9      // 于是在mixup方法中, 可以使用4个类型参数: X3, Y3, X2, Y2
10     fn mixup<X2, Y2>(self, other: Point<X2, Y2>) -> Point<X3, Y2> {
11         Point {
12             x: self.x,
13             y: other.y,
14         }
15     }
16 }
17
18 fn main() {
19     let p1 = Point { x: 5, y: 10.4 };
20     let p2 = Point { x: "Hello", y: 'c' };
21
22     let p3 = p1.mixup(p2);
23
24     println!("p3.x = {}, p3.y = {}", p3.x, p3.y);
25 }
26
27 // 输出
28 p3.x = 5, p3.y = c

```

可以看到，我们在 `Point<X3, Y3>` 的方法 `mixup()` 上新定义了两个类型参数 `X2`、`Y2`，于是在 `mixup()` 方法中，可以同时使用 4 个类型参数：`X2`、`Y2`、`X3`、`Y3`。你可以品味一下这个示例，不过这种复杂情况，现在的你只需要了解就可以了。

接下来我们来聊一聊 Rust 里的类型体系构建方法。

类型体系构建方法

类型体系构建指的是如何从最底层的小砖块开始，通过封装、组合、集成，最后修建成一座类型上的摩天大楼。在 Rust 里，主要有四大基础设施参与这个搭建的过程。

struct 结构体

enum 枚举

洋葱结构

type 关键字

struct 和 enum

struct 是 Rust 里把简单类型组合起来的主要结构。struct 里的字段可以是基础类型，也可以是其他结构体或枚举等复合类型，这样就可以一层一层往上套。结构体 struct 表达的是一种元素间同时起作用的结构。

而枚举 enum 表达的是一种元素间在一个时刻只有一种元素起作用的结构。因此枚举类型特别适合做配置和类型聚合之类的工作。

你可以看一下下面的综合示例，它是对某个学校的数学课程建模的层级模型。

 复制代码

```
1 struct Point(u32, u32); // 定义点
2
3 struct Rectangle { // 长方形由两个点决定
4     p1: Point,
5     p2: Point,
6 }
```



```
7 struct Triangle(Point, Point, Point); // 三角形由三个点组成
8
9 struct Circle(Point, u32); // 圆由点和半径组成
10
11 enum Shape { // 由枚举把长方形，三角形和圆形聚合在一起
12     Rectangle(Rectangle),
13     Triangle(Triangle),
14     Circle(Circle),
15 }
16
17 struct Axes; // 定义坐标
18
19 struct Geometry { // 几何学由形状和坐标组成
20     shape: Shape,
21     axes: Axes,
22 }
23
24 struct Algebra; // 定义代数
25
26 enum Level { // 定义学校的级别
27     Elementary, // 小学
28     Secondary, // 初中
29     High, // 高中
30 }
31
32 enum Course { // 数学要学习几何和代数，由枚举来聚合
33     Geometry(Geometry),
34     Algebra(Algebra),
35 }
36
37 struct MathLesson { // 定义数学课程，包括数学的科目和级别
38     math: Course,
39     level: Level,
40 }
41
```

请根据注释认真体会其中类型的层级结构。你甚至可以试着去编译一下上述代码，看看是不是可以编译通过。

偷偷告诉你答案：是可以通过的！

newtype

结构体还有一种常见的封装方法，那就是用单元素的元组结构体。比如定义一个列表类型 `struct List(Vec<u8>);`。它实际就是 `Vec<u8>` 类型的一个新封装，相当于给里面原来那种类型取了一个新名字，同时也把原类型的属性和方法等屏蔽起来了。

有时，你还可以看到没有具化类型参数的情形。

```
1 struct List<T>(Vec<T>);
```

[复制代码](#)

这种模式非常常见，于是业界给它取了个名字，叫做 `newtype` 模式，意思是用新的类型名字替换里面原来那个类型名字。

洋葱结构

Rust 中的类型还有另外一种构建方法——洋葱结构。我们来看一个示例，注意代码里的 `type` 关键字在这里的作用是把一个类型重命名，取了一个更短的名字。

```
1 // 你可以试着编译这段代码
2 use std::collections::HashMap;
3
4 type AAA = HashMap<String, Vec<u8>>;
5 type BBB = Vec<AAA>;
6 type CCC = HashMap<String, BBB>;
7 type DDD = Vec<CCC>;
8 type EEE = HashMap<String, DDD>;
```

[复制代码](#)

最后 `EEE` 展开就是这样的：

```
1 HashMap<String, Vec<HashMap<String, Vec<HashMap<String, Vec<u8>>>>>>>>>;
```

[复制代码](#)

可以看到，尖括号的层数很多，像洋葱一样一层一层的，因此叫洋葱类型结构。只要你开心，你可以把这个层次无限扩展下去。

我们再来看一个结合 `newtype` 和 `struct` 的更复杂的示例。

[复制代码](#)

```
1 use std::collections::HashMap;
2
3 struct AAA(Vec<u8>);
4 struct BBB {
5     hashmap: HashMap<String, AAA>
6 }
7 struct CCC(BBB);
8 type DDD = Vec<CCC>;
9 type EEE = HashMap<String, Vec<DDD>>;
10
11 最后，EEE展开就类似下面这样（仅示意，无法编译通过）
12 HashMap<String, Vec<Vec<CCC(BBB {hashmap: HashMap<String, AAA<Vec<u8>>>})>>>>
```

可以看到，洋葱结构在嵌套层级多了之后，展开是相当复杂的。

type 关键字

`type` 关键字很重要，它的作用是在洋葱结构表示太长了之后，把一大串类型的表达简化成一个简短的名字。在 Rust 中使用 `type` 关键字，可以使类型大厦的构建过程变得清晰可控。

`type` 关键字还可以处理泛型的情况，比如：

[复制代码](#)

```
1 type MyType<T> = HashMap<String, Vec<HashMap<String, Vec<HashMap<String, Vec<T>>>>
```

因为最里面那个是类型 `Vec<T>`，`T` 类型参数还在。因此给这个洋葱类型重命名的时候，需要把这个 `T` 参数带上，于是就变成了 `MyType<T>`。

这种写法在标准库里很常见，最佳示例就是关于各种 `Result*` 的定义。

在 `std::io` 模块里，取了一个与 `std::result::Result<T, E>` 同名的 `Result` 类型，把 `std::result::Result<T, E>` 定义简化了，具化其 `Error` 类型为 `std::io::Error`，同时仍保留了第一个类型参数 `T`。于是得到了 `Result<T>`。

 复制代码

```
1 pub type Result<T> = Result<T, Error>;
```

刚开始接触 Rust 的时候，你可能会对这种表达方式产生疑惑，其实道理就在这里。以后你在阅读 Rust 生态中各种库的源码时，也会经常遇到这种封装方式，所以我们要习惯它。

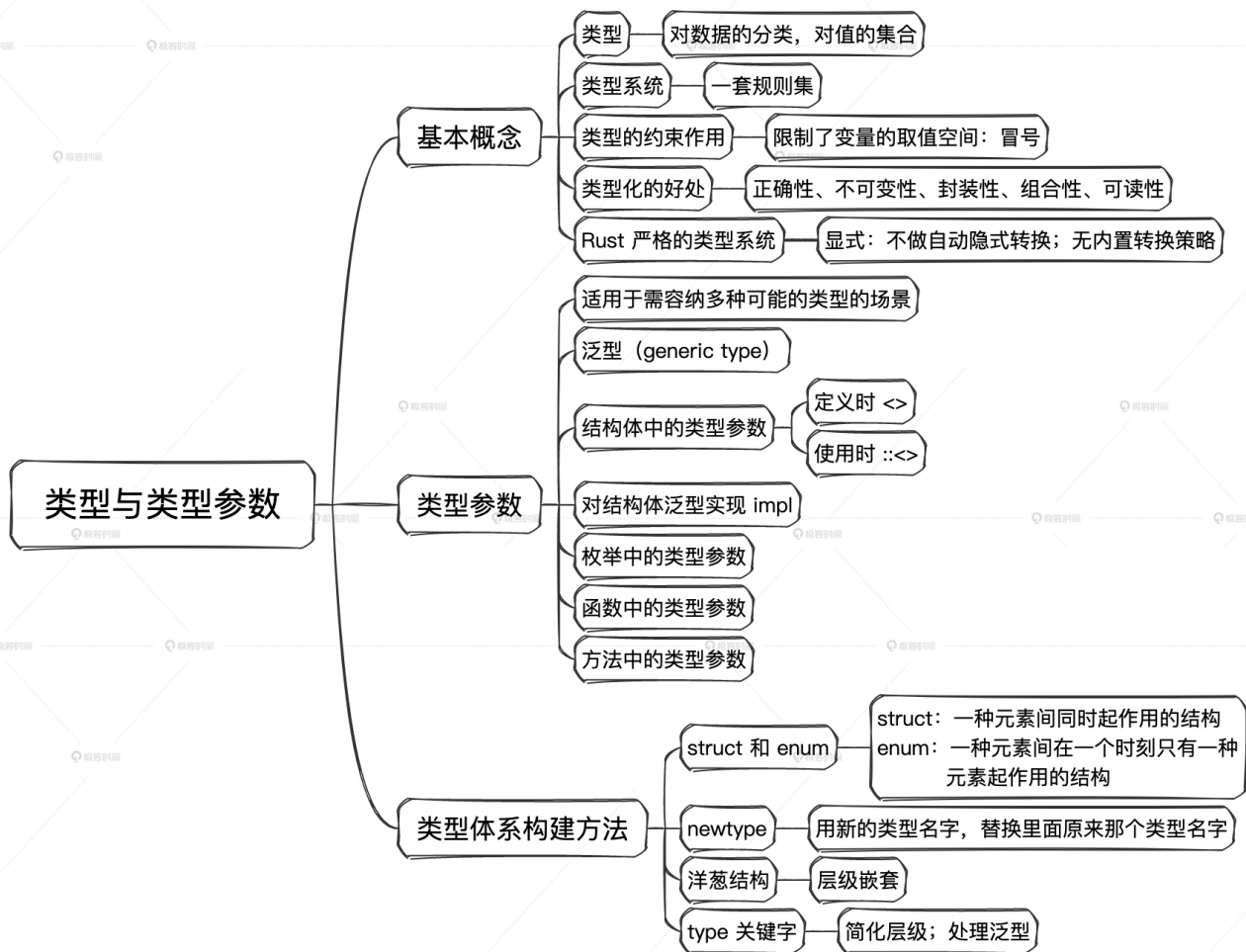
关于这种定义更多的资料参见：

[🔗 https://doc.rust-lang.org/std/result/enum.Result.html](https://doc.rust-lang.org/std/result/enum.Result.html)

[🔗 https://doc.rust-lang.org/std/io/type.Result.html](https://doc.rust-lang.org/std/io/type.Result.html)

[🔗 https://doc.rust-lang.org/std/io/struct.Error.html](https://doc.rust-lang.org/std/io/struct.Error.html)

小结



这节课我们对 Rust 中类型相关的知识做了一个专门的讲解。现代编程语言的趋势是越来越强调类型化，比如 TypeScript、Rust。一个成熟的类型系统对于编写健壮的程序来说至关重要。类型可以看作是对变量取值空间的一种约束。

在 Rust 中，有很多对多种类型做统一处理的需求，因此引入了类型参数和泛型的概念。它们实际是在类型化道路上的必然选择，因为单一的类型确实不方便，或者不能满足我们的需求。你要先克服对这种参数化设计的畏惧感，只要花一些时间熟悉这节课我们提到的那些形式，这些概念就不难掌握。

然后，这节课我们还讲解了 4 种类型体系建模方法，你可以在后面的实践过程中慢慢加深理解。

思考题

如果你给某个泛型实现了一个方法，那么，还能为它的一个具化类型再实现同样的方法吗？

欢迎你把思考后的结果分享到评论区，也欢迎你把这节课的内容分享给需要的朋友，我们下节课再见！

《Programming.with.Types》原文摘录

TYPE A *type* is a classification of data that defines the operations that can be done on that data, the meaning of the data, and the set of allowed values. Typing is checked by the compiler and/or run time to ensure the integrity of the data, enforce access restrictions, and interpret the data as meant by the developer.

In some cases, we will simplify our discussion and ignore the operations part, so we'll look at types simply as sets, which represent all the possible values an instance of that type can take.

TYPE SYSTEM A *type system* is a set of rules that assigns and enforces types to elements of a programming language. These elements can be variables, functions, and other higher-level constructs. Type systems assign types through notation you provide in the code or implicitly by deducing the type of a certain element based on context. They allow various conversions between types and disallow others.

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (6)

最新 精选



古明地觉

2023-11-03 来自中国台湾

思考题："为泛型实现了一个方法，能否再为具化类型实现一个同名方法"，取决于这个泛型能否表示相应的具化类型。比如为泛型 `T` 和 `String` 实现了相同的方法，由于 `T` 没有施加任何约束，它可以代表 `String`。那么当调用方法时，对于具化类型 `String` 来说，要调用哪一个呢？因此会出现歧义，编译器会报错：方法被重复定义了。

但如果给泛型 T 施加了一个 Copy 约束，要求 T 必须实现了 Copy trait，那么就不会报错了，因为此时 T 代表不了 String，所以调用方法不会出现歧义。但如果再为 i32 实现一个同名方法就会报错了，因为 i32 实现了 Copy，它可以被 T 表示。

PS：老师我在 06 讲提了一个问题，之前在学 Rust 的时候就一直困扰着我，还麻烦老师解答一下。

作者回复：👍



👍 5



一个人旅行

2023-11-03 来自北京

不能，编译器会提示duplicate definitions for XXXXX。

如果想为具化类型再实现同样的方法，则可以定义一个trait，用具化类型实现这个trait，来达到"为具化类型再实现同样的方法"的目的。

作者回复：👍



👍 3



Geek_de05b4

2023-11-03 来自上海

```
impl<T: std::ops::Add<Output = T>> Bar<T> {  
    fn plus(self, other: Bar<T>) -> T {  
        self.value + other.value  
    }  
}
```

```
impl Bar<u32> {  
    fn plus(&self, other: &Bar<u32>) -> u32 {  
        self.value + other.value  
    }  
}
```

//Output:

error[E0592]: duplicate definitions with name `plus`

```
--> src/main.rs:64:5
|
64 |   fn plus(self, other: Bar<T>) -> T {
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ duplicate definitions for `plus`
...
70 |   fn plus(&self, other: &Bar<u32>) -> u32 {
|   ----- other definition for `plus`
```

不能

作者回复: 👍



👍 1



Noah • ^ •

2023-11-09 来自浙江

思考题：通常是不能，但是可以通过 trait 进行特化

作者回复: 对的



-Hedon 🌈

2023-11-08 来自湖北

思考题：核心点就在于不能为同一个类型实现 2 个相同函数签名的方法，因为这会引起方法冲突。

编译报错如下：

```
error[E0592]: duplicate definitions with name `print`
--> examples/generic.rs:7:5
|
7 |   fn print(&self) {
|   ^^^^^^^^^^^^^^^^^ duplicate definitions for `print`
...
13 |   fn print(&self) {
|   ----- other definition for `print`
```


所以一般情况下，如果 `impl<T>` 后面的 `T` 没有任何的约束，那么就表示为所有类型的 `T` 都实现了方法，比如说 `print()`，这个时候是不能为具化类型再次实现 `print()` 的，因为这个时候就产生了方法冲突。

但是，如果 `impl<T: std::fmt::Display>` 后面的 `T` 是有约束的，那么其实只为符合这个约束的类型实现了 `print()`，其余类型是没有实现的，所以是可以为其余具化类型实现相同的方法的。

如：

```
```rust
struct Point<T> {
 x: T,
 y: T,
}

struct NotDisplay {
 a: u32,
}

impl<T: std::fmt::Display> Point<T> {
 fn print(&self) {
 println!("Point: {}, {}", self.x, self.y);
 }
}

impl Point<NotDisplay> {
 fn print(&self) {
 println!("not display");
 }
}
```
```

作者回复：很棒





伯阳

2023-11-03 来自北京

和Java的范型类似



1